

Arquitectura y Servicios de Red
ISIS AYSR – 6L
Colombia, Bogotá

Laboratory No.05 – Databases and Network Protocols

Oscar Andres Sanchez Porras
Felipe Amador Gonzales
Diego Fabian Andrade

ABSTRACT

“This laboratory focuses on the review of network protocols through Wireshark captures and on the installation and configuration of database management systems (DBMS) in multiple environments. DNS message analysis is performed by examining name resolution for www.google.com, including TCP/IP fields and payload. HTTP messages and Ethernet frame headers are examined for connections to a university web server. Regarding database systems, SQL Server 2025 is installed on Windows Server, PostgreSQL is configured on Linux Slackware, and Azure SQL Database is used as a cloud service. For each engine, users are created, databases with at least three related tables are designed, data is inserted, and remote connections are established using DBeaver. Additionally, conceptual questions about Microsoft Azure cloud computing are addressed, covering service models (IaaS, PaaS, SaaS), availability zones, scaling strategies, and TLS security.”

Index Terms: DNS, HTTP, Ethernet, Wireshark, SQL Server, PostgreSQL, Azure SQL, DBeaver, cloud computing, IaaS, PaaS, SaaS, TLS.

Contents

1	Introduction	1
2	Objective	

3	Parte 1: Review of Network Protocols	2
3.1	Revisión de Mensajes DNS	2
3.2	Revisión de Mensajes HTTP	3
3.3	Revisión de Tramas Ethernet	3
4	Parte 2: Base Software Installation	4
4.1	SQL Server – Windows Server	4
4.2	Azure SQL Database – Microsoft Azure	5
4.2.1	¿Qué es la computación en la nube y ventajas de Azure?	5
4.2.2	Tipos de servicios: IaaS, PaaS y SaaS	5
4.2.3	Regiones y zonas de disponibilidad	5
4.2.4	Escalado vertical vs. horizontal	6
4.2.5	TLS en la capa de transporte: Azure SQL vs. VM local	6
4.3	PostgreSQL – Linux Slackware	7
4.4	Other Database Engine Configurations – DBeaver	8
5	Conclusiones	8
6	Anexos	9
	Referencias	9

1. INTRODUCTION

1	La infraestructura de TI de una empresa moderna combina
2	múltiples capas de servicios: desde la capa física y de red

hasta los sistemas de gestión de bases de datos y servicios en la nube. Comprender cómo interactúan los protocolos de red en cada capa, así como saber instalar y administrar motores de bases de datos, es fundamental para cualquier profesional de sistemas y redes.

En este quinto laboratorio se trabaja en dos frentes complementarios. Por un lado, se profundiza en el análisis de protocolos de red mediante capturas de Wireshark: se examinan mensajes DNS generados al acceder a www.google.com, se analizan los encabezados HTTP de peticiones a un servidor universitario y se estudia la estructura de las tramas Ethernet que encapsulan dichas comunicaciones.

Por otro lado, se instalan y configuran tres motores de bases de datos: SQL Server 2025 sobre Windows Server, PostgreSQL sobre Linux Slackware y Azure SQL Database como servicio en la nube. En cada entorno se crean usuarios, bases de datos con al menos tres tablas relacionadas, se insertan datos de prueba y se establecen conexiones remotas mediante el cliente DBeaver.

2. OBJECTIVE

Continuar aprendiendo la instalación de software base, específicamente motores de gestión de bases de datos (SQL Server, PostgreSQL y Azure SQL), y revisar el funcionamiento de los protocolos de red (DNS, HTTP y Ethernet) mediante capturas con Wireshark en un entorno de infraestructura empresarial simulada.

3. PARTE 1: REVIEW OF NETWORK PROTOCOLS

3.1 Revisión de Mensajes DNS

Con Wireshark se realizó una consulta a la página web <http://www.google.com/>, filtrando los mensajes DNS para examinar la resolución de nombres.

Filtro usado: `dns.qry.name matches "www.google.com"`

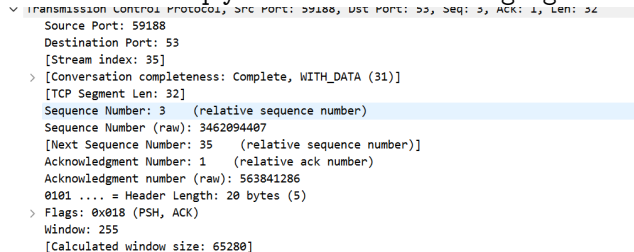


Fig. 1. Captura Wireshark – mensajes DNS para www.google.com.

Flujo general de la comunicación:

Se observa el siguiente patrón típico de resolución DNS:

Cliente → Servidor DNS

- **IP origen:** 192.168.20.21 (equipo local)

- **IP destino:** 190.157.8.101 (servidor DNS)
- **Puerto destino:** 53 (DNS)
- **Protocolo:** DNS sobre TCP (no UDP)

Consultas realizadas:

- **Tipo A** → IPv4
- **Tipo AAAA** → IPv6
- **Tipo HTTPS** → registros modernos (HTTPSSVC)

Respuesta del Servidor DNS → Cliente:

El servidor responde con las siguientes direcciones:

- **IPv4:** 142.251.155.119, 142.251.150.119, 142.251.154.x
- **IPv6:** 2001:4860:482d:7700::...

Esto confirma que www.google.com se resuelve correctamente tanto en IPv4 como en IPv6.

Detalles del paquete (TCP/IP):

TCP:

- Puerto origen: 59188
- Puerto destino: 53
- Flags: PSH, ACK
- Secuencia relativa: 3 / Ack: 1
- Ventana: 255 (calculada: 65280)

Indica que la conexión TCP ya está establecida y se están enviando datos (consulta DNS).

IP:

- Versión: IPv4
- TTL: 128 → típico de Windows
- Longitud total: 72 bytes
- Flags: Don't Fragment

Reensamblado TCP:

El campo *Reassembled TCP Segments* indica que la consulta DNS fue fragmentada en varios segmentos TCP y Wireshark los reconstruyó correctamente. Esto es común cuando el tamaño del mensaje DNS es grande o se usan extensiones modernas como HTTPS records.

Payload (datos):

En el panel hexadecimal se observa el texto www.google.com, confirmando que el contenido del paquete es efectivamente la consulta DNS.

DNS sobre TCP:

Normalmente DNS usa UDP (puerto 53), pero en esta captura se usa TCP. Las posibles razones son: respuestas grandes por uso de DNSSEC o múltiples registros, extensiones modernas como HTTPS records, o configuración específica del sistema o del servidor DNS. El uso de TCP es válido pero menos común.

Conclusión:

- El equipo (192.168.20.21) realiza correctamente consultas DNS.
- El servidor DNS (190.157.8.101) responde adecuadamente.

- Se obtienen múltiples IPs (balanceo de carga de Google).
- Se usa TCP en lugar de UDP, lo cual es válido pero menos frecuente.
- No se observan errores en la resolución.

3.2 Revisión de Mensajes HTTP

Con Wireshark se realizó una consulta a <http://profesores.is.escuelaing.edu.co/~csantiago/> y se filtraron los mensajes HTTP para examinar el encabezado.

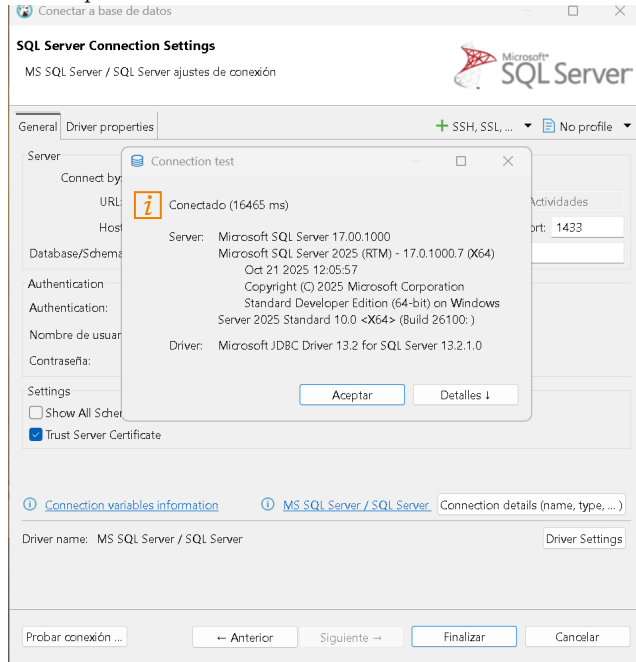


Fig. 2. Captura Wireshark – mensajes HTTP (GET y 200 OK).

1. Solicitud HTTP (GET):

En la captura aparece:

```
GET /~csantiago/ HTTP/1.1
```

Esto indica que el navegador realizó correctamente una petición HTTP GET al servidor. La ruta solicitada coincide con la URL del laboratorio.

2. Respuesta del servidor:

Se observa también:

```
HTTP/1.1 200 OK (text/html)
```

El servidor respondió exitosamente y entregó el contenido de la página web. Se tienen ambas partes del intercambio: el GET (cliente → servidor) y el 200 OK (servidor → cliente).

3. Otros paquetes observados (normales):

- GET /~csantiago/foto.jpg → el navegador descarga recursos adicionales.
- GET /favicon.ico → solicitud automática del navegador.
- 304 Not Modified → uso de caché del navegador.

4. Análisis TCP:

- Src Port: 80 → servidor HTTP
- Dst Port: 53698 → cliente
- Flags: PSH, ACK

Esto confirma que es tráfico HTTP sobre TCP con la comunicación cliente-servidor correctamente establecida.

3.3 Revisión de Tramas Ethernet

Con Wireshark se filtró por mensajes GET (`http.request.method == GET`) para examinar el encabezado de la trama Ethernet.

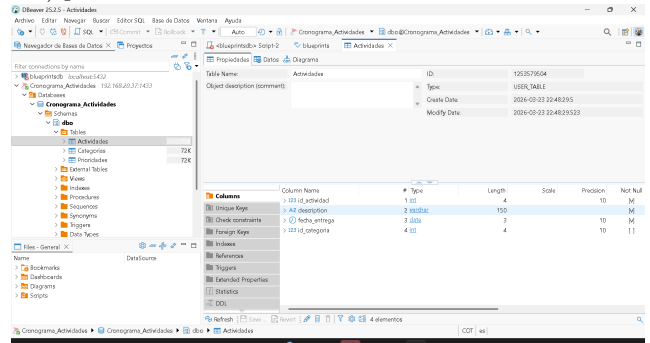


Fig. 3. Captura Wireshark – encabezado trama Ethernet.

1. Identificación de la trama:

- Frame 47219 (ejemplo)
- Tipo: Ethernet II
- Longitud: 789 bytes

El paquete HTTP GET está encapsulado en: Ethernet → IP → TCP → HTTP.

2. Estructura del encabezado Ethernet:

TABLA I

CAMPOS DE LA TRAMA ETHERNET II

Campo	Tamaño
MAC destino	6 bytes
MAC origen	6 bytes
Tipo (EtherType)	2 bytes
Total encabezado	14 bytes

3. Análisis específico de la captura:

```
Ethernet II,
Src: Intel_01:30:ad (fc:44:82:d1:30:ad),
Dst: HefeiRadioCo_00:02:0e
(14:82:5b:e0:00:02)
```

- **MAC origen:** fc:44:82:d1:30:ad – Fabricante: Intel. Representa el equipo local (192.168.20.21).
- **MAC destino:** 14:82:5b:e0:00:02 – Fabricante: Hefei Radio Communication. Esta es la MAC del **gateway/router local**, no la del servidor web, ya que el destino IP está fuera de la red local.
- **EtherType:** 0x0800 → IPv4. Indica que el payload es un paquete IP.

4. Flujo completo de encapsulación:

```
[Ethernet] Src MAC: PC (Intel)
          Dst MAC: Router
|
[IP] Src: 192.168.20.21
    Dst: 45.239.88.86
|
[TCP] Dst Port: 80
|
[HTTP] GET /~csantiago/ HTTP/1.1
```

5. Observaciones importantes:

- **MAC** se usa para comunicación local (LAN); **IP** para comunicación global (Internet). Por eso la MAC destino es la del router y la IP destino es la del servidor web.
- El FCS (CRC) no aparece en Wireshark porque la NIC lo descarta antes de pasar el frame al sistema operativo.

Conclusión: El encabezado Ethernet confirma que el cliente envía la solicitud al router local, el tipo de trama es IPv4 y se evidencia el proceso de encapsulación por capas de forma correcta.

4. PARTE 2: BASE SOFTWARE INSTALLATION

4.1 SQL Server – Windows Server

Se instaló el DBMS SQL Server 2025 Standard Developer Edition en la máquina virtual con Windows Server. Se seleccionó la opción de instalación **Básica** para instalar el motor con la configuración predeterminada.

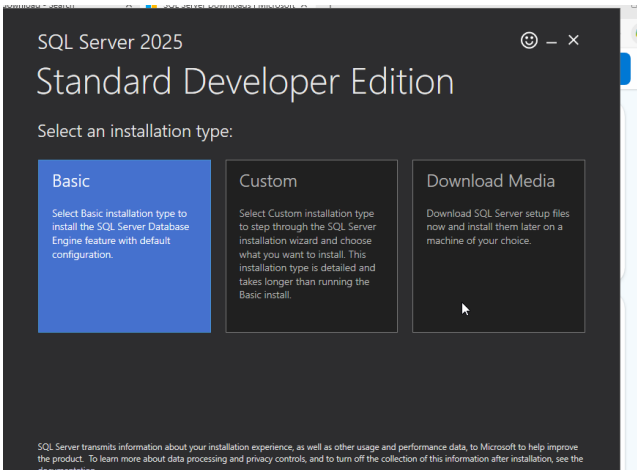


Fig. 4. Instalación de SQL Server 2025 – opción Básica.

Creación de usuarios:

Se crearon tres usuarios con autenticación SQL: **Oscar**, **Felipe** y **Fabian**, con contraseña **Admin2026***.

```
1 CREATE TABLE Categorías(
2     id_categorias INT PRIMARY KEY IDENTITY(1,1),
3     nombre_categoria VARCHAR(50) NOT NULL
4 );
5 CREATE TABLE Actividades (
6     id_actividad INT PRIMARY KEY IDENTITY(1,1),
7     description VARCHAR(150) NOT NULL,
8     fecha_entrega DATE NOT NULL,
9     id_categoria INT,
10    CONSTRAINT FK_Categoria FOREIGN KEY (id_categoria)
11    REFERENCES Categorías(id_categorias)
12 );
```

Fig. 5. Logins creados en SQL Server: Oscar, Felipe y Fabian.

Creación de base de datos y tablas:

Se creó la base de datos **Cronograma_Actividades** con tres tablas relacionadas: **Categorías**, **Actividades** y **Prioridades**. Adicionalmente, se asignó acceso exclusivo al usuario **Oscar** mediante el rol **db_owner**.

```
CREATE DATABASE Cronograma_Actividades;
GO
USE Cronograma_Actividades;
GO

CREATE TABLE Categorías(
    id_categorias INT PRIMARY KEY
    IDENTITY(1,1),
    nombre_categoria VARCHAR(50) NOT NULL
);

CREATE TABLE Actividades(
    id_actividad INT PRIMARY KEY
    IDENTITY(1,1),
    description VARCHAR(150) NOT NULL,
    fecha_entrega DATE NOT NULL,
    id_categoria INT,
    CONSTRAINT FK_Categoria FOREIGN KEY
    (id_categoria)
    REFERENCES Categorías(id_categorias)
);

CREATE TABLE Prioridades(
    id_prioridad INT PRIMARY KEY
    IDENTITY(1,1),
    nivel_prioridad VARCHAR(20) NOT NULL,
    id_actividad INT,
    CONSTRAINT FK_Actividad FOREIGN KEY
    (id_actividad)
    REFERENCES Actividades(id_actividad)
);

USE Cronograma_Actividades;
GO
ALTER ROLE db_owner ADD MEMBER Oscar;
GO
```

Listing 1: Creación de BD y tablas en SQL Server

id_actividad	description	fecha_entrega	id_categoria
1	Laboratorio Abans SQL	2026-03-24	1
2	Proyecto SIRHA UI	2026-03-28	1

Fig. 6. Creación de tablas y asignación de rol a Oscar.

Población de tablas y verificación:

Se insertaron datos en las tres tablas y se verificó su contenido con sentencias `SELECT * FROM`. Los resultados muestran:

- **Actividades:** Entrega Laboratorio 5 SQL (2026-03-25), Estudiar para parcial de Redes (2026-03-24), Reunión de grupo Laboratorio (2026-03-24).
- **Categorías:** Académico, Personal, Proyectos.
- **Prioridades:** ALTA (act. 1), MEDIA (act. 2), ALTA (act. 3).

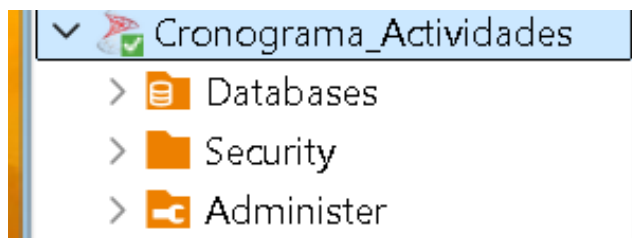


Fig. 7. Resultado `SELECT` de las tres tablas pobladas.

Se confirma además que únicamente el usuario **Oscar** aparece en la sección *Users* de la base de datos *Cronograma_Actividades*, validando el acceso exclusivo configurado.

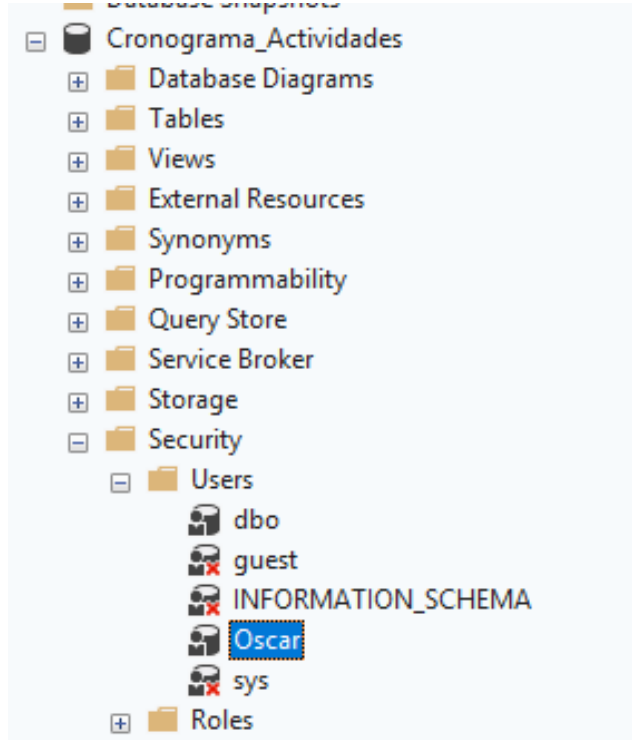


Fig. 8. Solo el usuario Oscar tiene acceso a Cronograma_Actividades.

4.2 Azure SQL Database – Microsoft Azure

Se utilizó el portal de Microsoft Azure con la suscripción *Azure for Students* para desplegar una base de datos SQL en la nube.

Preguntas conceptuales sobre Azure:

4.2.1 ¿Qué es la computación en la nube y ventajas de Azure?

La computación en la nube consiste en ofrecer recursos y servicios tecnológicos —como servidores, almacenamiento, bases de datos, redes, software y herramientas de análisis— a través de Internet. Este modelo permite acceder a ellos de forma flexible, escalable y según la demanda, sin necesidad de contar con infraestructura física propia.

En comparación con una infraestructura local, Microsoft Azure presenta las siguientes ventajas:

- **Menor costo operativo:** Se evitan inversiones en equipos físicos, mantenimiento y consumo energético.
- **Escalabilidad dinámica:** Los recursos se amplían o reducen según las necesidades, pagando solo por lo utilizado.
- **Alta disponibilidad:** Ofrece sistemas de respaldo, redundancia y recuperación ante fallos o desastres.
- **Actualizaciones gestionadas:** Microsoft aplica parches, mejoras y medidas de seguridad automáticamente.
- **Cobertura global:** Sus servicios están distribuidos en múltiples regiones para un acceso rápido y eficiente.

4.2.2 Tipos de servicios: IaaS, PaaS y SaaS

Microsoft Azure ofrece tres modelos principales de servicios en la nube, cada uno con distintos niveles de control y gestión:

- **IaaS** (Infraestructura como Servicio): Recursos virtuales como máquinas, redes y almacenamiento. El usuario tiene mayor control. Ej: Azure Virtual Machines.
- **PaaS** (Plataforma como Servicio): Entorno listo para desarrollar y desplegar aplicaciones sin gestionar la infraestructura. Ej: Azure App Services.
- **SaaS** (Software como Servicio): Aplicaciones completas accesibles desde internet sin instalación. Ej: Microsoft 365, Azure DevOps.

La principal diferencia radica en el nivel de control: IaaS es alto (el usuario gestiona casi todo), PaaS es intermedio (Azure gestiona la infraestructura) y SaaS es bajo (todo administrado por el proveedor).

4.2.3 Regiones y zonas de disponibilidad

En Microsoft Azure, las **regiones** son áreas geográficas con centros de datos interconectados que trabajan de forma redundante. Las **zonas de disponibilidad** son ubicaciones separadas dentro de una misma región, diseñadas para aislar fallos, con su propia infraestructura de energía y red.

Su impacto en la disponibilidad del servicio es el siguiente:

- **Mayor disponibilidad:** Al distribuir aplicaciones entre varias zonas, se evita que un fallo en un centro de datos afecte todo el sistema.
- **Mejor rendimiento:** Elegir regiones cercanas al usuario reduce la latencia.
- **Cumplimiento normativo:** Permiten almacenar datos en ubicaciones específicas según regulaciones legales.

4.2.4 Escalado vertical vs. horizontal

- **Escalado vertical:** Aumenta la capacidad de un solo recurso (ej: pasar de 4 a 8 vCPU en una VM). Adecuado para aplicaciones monolíticas o bases de datos tradicionales que dependen de un solo servidor.
- **Escalado horizontal:** Agrega más instancias del mismo recurso (ej: usar AKS para añadir nodos a un clúster). Ideal para aplicaciones modernas distribuidas y micro-servicios, ya que mejora la disponibilidad y reparte la carga.

4.2.5 TLS en la capa de transporte: Azure SQL vs. VM local

El uso de TLS influye directamente en cómo se protegen los datos entre el cliente y la base de datos:

- **Azure SQL Database:** Requiere TLS de forma obligatoria. No permite deshabilitar el cifrado. Los certificados y configuraciones de seguridad son gestionados automáticamente por Azure (TLS 1.2 o superior). Incorpora firewalls a nivel de servicio y un gateway que distribuye las conexiones.
- **Base de datos en VM local:** TLS es opcional y depende de la configuración del administrador. Ofrece mayor control sobre protocolos TCP/IP y puertos, pero requiere configurar manualmente firewalls, reglas de acceso y NAT. No cuenta con una capa intermedia de gestión de conexiones.

En consecuencia, Azure garantiza la protección de los datos sin intervención del usuario, mientras que en un entorno local todo depende de una configuración manual adecuada.

Despliegue de Azure SQL Database:

Se creó el recurso `Microsoft.SqlDatabase.newDatabaseNewServer` en el grupo de recursos `laboratorio8` bajo la suscripción *Azure for Students*. La implementación se completó exitosamente.

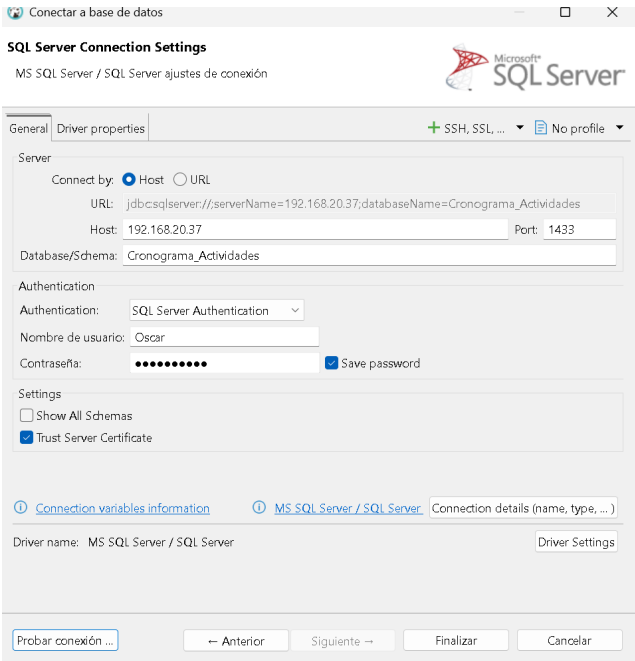


Fig. 9. Implementación completada en Azure – grupo laboratorio8.

Se accedió a la base de datos `oscar-laboratorio` mediante el Editor de Power Query del portal de Azure usando autenticación SQL con el usuario `oscar`.

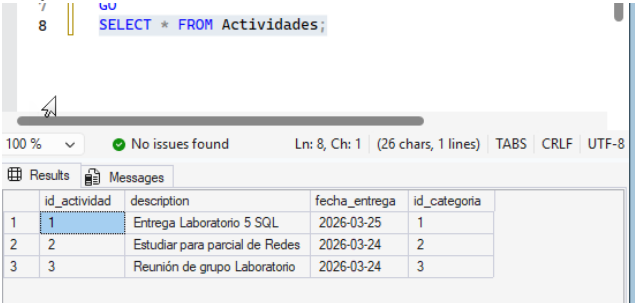


Fig. 10. Acceso a Azure SQL Database – autenticación SQL.

Creación y población de tablas en Azure:

Se crearon las tablas `Categorias`, `Actividades` y `Prioridades` con la misma estructura que en SQL Server local, y se poblaron con datos iniciales:

```
-- Insertar datos iniciales
INSERT INTO Categorias (nombre_categoria)
VALUES ('Universidad'), ('Personal');

INSERT INTO Actividades
(description, fecha_entrega, id_categoria)
VALUES
('Laboratorio Azure SQL', '2026-03-24', 1),
('Proyecto SIRHA UI', '2026-03-28', 1);

INSERT INTO Prioridades
(nivel_prioridad, id_actividad)
VALUES ('ALTA', 1), ('MEDIA', 2);
```

Listing 2: Inserción de datos en Azure SQL

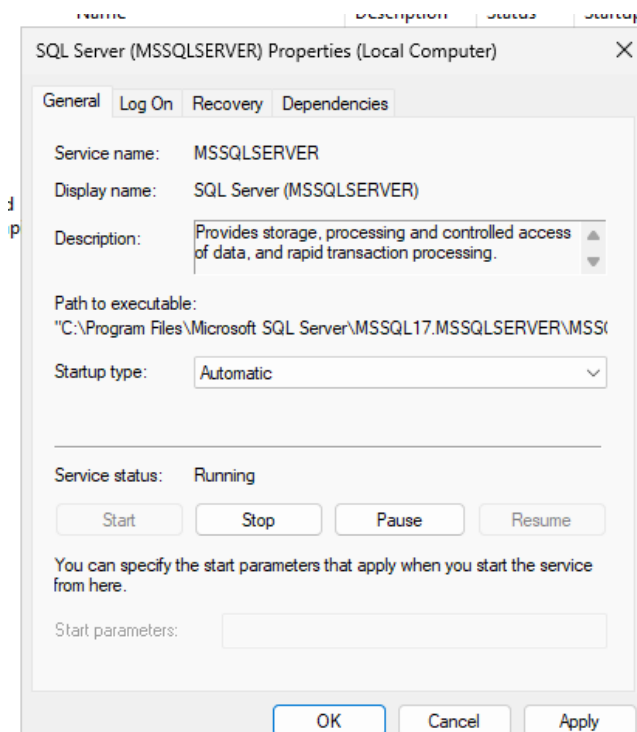


Fig. 11. Creación de tablas en Azure SQL Database.



No hay Recursos de Azure SQL para mostrar

Intente cambiar o borrar los filtros.

[Más información](#)

Fig. 12. Tablas pobladas y consultadas con SELECT en Azure.

Eliminación de recursos:

Finalmente, se eliminó el grupo de recursos laboratorio8 completo para evitar costos adicionales y el agotamiento de créditos disponibles.

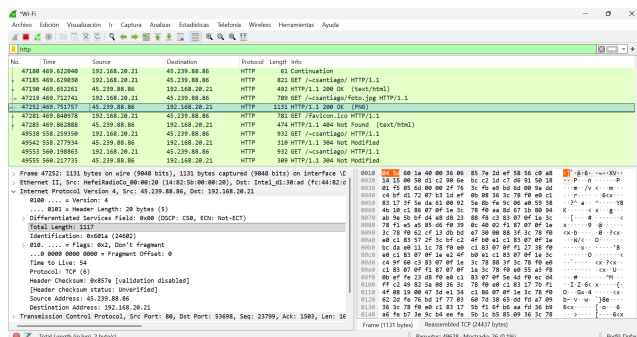


Fig. 13. Eliminación del grupo de recursos laboratorio8 en Azure.

4.3 PostgreSQL – Linux Slackware

Se instaló y configuró el DBMS PostgreSQL en la máquina virtual con Linux Slackware. La base de datos diseñada almacena información sobre sitios turísticos de Colombia.

Inicio del servidor PostgreSQL:

Se accedió al usuario postgres y se inició el servidor con pg_ctl, indicando la ruta de datos /usr/local/pgsql/data. Finalmente, se accedió al cliente mediante el comando psql.

```
su - postgres
/usr/local/pgsql/bin/pg_ctl -D \
  /usr/local/pgsql/data -l logfile start
/usr/local/pgsql/bin/psql
```

Listing 3: Inicio de PostgreSQL en Slackware

```
root@darkstar:~# su - postgres
A horse! A horse! My kingdom for a horse!
-- Wm. Shakespeare, "Henry VI"

postgres@darkstar:~$ /usr/local/pgsql/bin/pg_ctl -D /usr/local/pgsql/data -l logfile start
pg_ctl: another server might be running; trying to start server anyway
waiting for server to start.... done
server started
postgres@darkstar:~$ /usr/local/pgsql/bin/psql
psql (6.2)
Type "help" for help.
```

Fig. 14. Inicio del servidor PostgreSQL en Slackware.

Conexión a la base de datos y listado de tablas:

Se conectó a la base de datos turismo_prueba1 mediante el comando \c y se listaron las tablas existentes con \dt:

```
postgres=# \c turismo_prueba1
postgres=# \dt
```

Como resultado, se observan las tablas ciudades, sitios y visitas, confirmando que la estructura de la base de datos fue implementada correctamente.

```
postgres=# \c turismo_prueba1
You are now connected to database "turismo_prueba1" as user "postgres".
turismo_prueba1=# \dt
          List of relations
Schema | Name      | Type  | Owner
-----+-----+-----+-----
public | ciudades  | table | postgres
public | sitios    | table | postgres
public | visitas   | table | postgres
(3 rows)
```

Fig. 15. Listado de tablas en turismo_prueba1.

Consulta de registros:

Se consultaron los registros de las tres tablas con SELECT * FROM:

```
SELECT * FROM ciudades;
-- id | nombre | departamento
-- 1 | Bogota | Cundinamarca
-- 2 | Medellin | Antioquia

SELECT * FROM sitios;
-- id | nombre | tipo | id_ciudad
-- 1 | Monserrate | Montana | 1
-- 2 | Guatape | Turistico | 2

SELECT * FROM visitas;
-- id | fecha | calificacion | id_sitio
-- 1 | 2026-03-20 | 5 | 1
```

```
turismo_prueba1=# SELECT * FROM ciudades;
```

id	nombre	departamento
1	Bogota	Cundinamarca
2	Medellin	Antioquia

(2 rows)

```
turismo_prueba1=# SELECT * FROM sitios;
```

id	nombre	tipo	id_ciudad
1	Monserate	Montaña	1
2	Guatape	Turistico	2

(2 rows)

```
turismo_prueba1=# SELECT * FROM visitas;
```

id	fecha	calificacion	id_sitio
1	2026-03-20	5	1

(1 row)

Fig. 16. Registros en tablas ciudades, sitios y visitas.

Los resultados verifican la relación entre las tablas: los sitios turísticos están asociados a ciudades y las visitas están asociadas a los sitios registrados. La base de datos fue creada y poblada exitosamente.

4.4 Other Database Engine Configurations – DBeaver

Se configuraron los motores de bases de datos para iniciarse automáticamente con el sistema operativo. Adicionalmente, se estableció una conexión remota a las bases de datos desde una máquina externa usando el cliente DBeaver.

Verificación del puerto SQL Server:

Se verificó la conectividad al servidor SQL Server en el puerto 1433 usando PowerShell:

```
PS> tnc 192.168.20.37 -port 1433
```

```
ComputerName      : 192.168.20.37
RemoteAddress     : 192.168.20.37
RemotePort        : 1433
InterfaceAlias    : Wi-Fi
SourceAddress     : 192.168.20.21
TcpTestSucceeded  : True
```

Fig. 17. Prueba de conectividad TCP al puerto 1433 (SQL Server).

Conexión exitosa con DBeaver:

Se configuró DBeaver con los siguientes parámetros para conectarse a la base de datos Cronograma_Actividades:

- **Host:** 192.168.20.37 – **Puerto:** 1433
- **Base de datos:** Cronograma_Actividades
- **Autenticación:** SQL Server Authentication
- **Usuario:** Oscar

La prueba de conexión confirmó: *Conectado (16465 ms)* – Microsoft SQL Server 2025 (RTM) – 17.0.1000.7 (X64).

Fig. 18. Conexión exitosa a SQL Server desde DBeaver.

Fig. 19. Explorador de DBeaver – base de datos Cronograma_Actividades.

Se visualizaron las tres tablas creadas (Actividades, Categorias, Prioridades) con sus columnas, tipos de datos, claves primarias y claves foráneas correctamente definidas.

Fig. 20. Estructura de tablas visualizada en DBeaver.

5. CONCLUSIONES

El análisis con Wireshark permitió comprender en detalle el comportamiento de los protocolos DNS, HTTP y Ethernet. Se pudo verificar que una consulta DNS a [www.google.com](http://profesores.is.escuelaing.edu.co/~csantiago/) devuelve múltiples registros A y AAAA usando TCP sobre el puerto 53, con un TTL de 128 que confirma el origen Windows. La petición HTTP GET a <http://profesores.is.escuelaing.edu.co/~csantiago/> generó una respuesta 200 OK y el análisis de la trama Ethernet confirmó que la MAC destino co-

rresponde al router local, evidenciando el proceso correcto de encapsulación por capas.

Respecto a los motores de bases de datos, se instaló y configuró SQL Server 2025 en Windows Server, PostgreSQL en Linux Slackware y Azure SQL Database en la nube, creando en cada caso bases de datos con al menos tres tablas relacionadas y datos de prueba. La conexión remota con DBeaver a SQL Server fue exitosa, confirmando la correcta configuración del servicio y su accesibilidad en la red.

Las preguntas conceptuales sobre Azure evidencian que la nube ofrece ventajas significativas en escalabilidad, disponibilidad y seguridad (TLS obligatorio, firewalls integrados) en comparación con una infraestructura local, donde la responsabilidad de configuración recae completamente en el administrador.

Lecciones aprendidas:

- DNS puede operar sobre TCP cuando las respuestas son demasiado grandes para UDP.
- La MAC destino de una trama siempre es la del siguiente salto (router), no la del destino IP final.
- Azure SQL Database impone TLS obligatorio, lo que elimina la posibilidad de conexiones inseguras.
- DBeaver requiere que el servicio SQL Server tenga habilitado el protocolo TCP/IP y el puerto 1433 abierto en el firewall.

6. ANEXOS

Gestor de contraseñas

Credenciales de acceso para todas las máquinas virtuales y bases de datos del laboratorio:

TABLA II
CREDENCIALES DE ACCESO

Sistema / Servicio	Usuario	Contraseña
Ubuntu (root)	root	admin
Slackware (root)	root	admin
Android	--	admin
Ubuntu (usuario)	laboratorio1	Admin2026*
Windows Server	Administrator	Admin2026*
SQL Server (usuarios)	Oscar / Felipe / Fabian	Admin2026*
Azure SQL	oscar	Admin2026*

REFERENCIAS

[1] Wireshark Foundation, *Wireshark User's Guide*, 2023. Disponible en: https://www.wireshark.org/docs/wsug_html_chunked/

[2] Microsoft, *SQL Server 2025 Documentation*, 2025. Disponible en: <https://learn.microsoft.com/en-us/sql/sql-server/>

[3] Microsoft, *Azure SQL Database Documentation*, 2025. Disponible en: <https://learn.microsoft.com/en-us/azure/azure-sql/>

[4] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2023. Disponible en: <https://www.postgresql.org/docs/16/>

[5] DBeaver Corp., *DBeaver User Guide*, 2024. Disponible en: <https://dbeaver.com/docs/dbeaver/>

[6] P. Mockapetris, "Domain Names – Implementation and Specification," RFC 1035, IETF, 1987.

[7] R. Fielding *et al.*, "Hypertext Transfer Protocol – HTTP/1.1," RFC 2616, IETF, 1999.